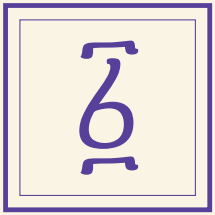
GEPA

Reflective prompt evolution (Agrawal et al., ICLR 2026)

Carlos Cotrini
AI in Industry, ETH Zurich
Weekend 1, October 2026



Section 1

The problem

GSM8K: grade-school maths word problems

Q: Alannah, Beatrix, and Queen are preparing for the new school year and have been given books by their parents. Alannah has 20 more books than Beatrix. Queen has 1/5 times more books than Alannah. If Beatrix has 30 books, how many books do the three have together?

Yang et al. 2024, Fig. 3 and section 5

GSM8K: grade-school maths word problems

Q: Alannah, Beatrix, and Queen are preparing for the new school year and have been given books by their parents. Alannah has 20 more books than Beatrix. Queen has 1/5 times more books than Alannah. If Beatrix has 30 books, how many books do the three have together?

The answer on file

140

Yang et al. 2024, Fig. 3 and section 5

GSM8K: grade-school maths word problems

Q: Alannah, Beatrix, and Queen are preparing for the new school year and have been given books by their parents. Alannah has 20 more books than Beatrix. Queen has 1/5 times more books than Alannah. If Beatrix has 30 books, how many books do the three have together?

A: The instruction goes here Then the working, then a number

At the start of the answer

The answer on file

140

Yang et al. 2024, Fig. 3 and section 5

GSM8K: grade-school maths word problems

Q: Alannah, Beatrix, and Queen are preparing for the new school year and have been given books by their parents. Alannah has 20 more books than Beatrix. Queen has 1/5 times more books than Alannah. If Beatrix has 30 books, how many books do the three have together?

A: The instruction goes here Then the working, then a number

At the start of the answer

The answer on file

140

Right if they match

Yang et al. 2024, Fig. 3 and section 5

GSM8K: grade-school maths word problems

Q: Alannah, Beatrix, and Queen are preparing for the new school year and have been given books by their parents. Alannah has 20 more books than Beatrix. Queen has 1/5 times more books than Alannah. If Beatrix has 30 books, how many books do the three have together?

A: The instruction goes here Then the working, then a number

At the start of the answer

The answer on file

140

Right if they match



7,473 training problems

To optimise on (a 3.5% sample)



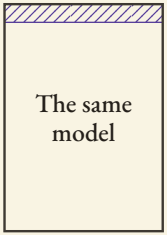
1,319 test problems

To check the result on

Yang et al. 2024, Fig. 3 and section 5

Same model, different instruction

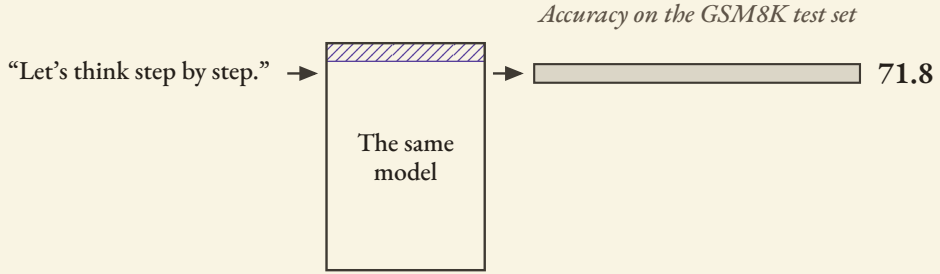
Accuracy on the GSM8K test set



PaLM 2-L

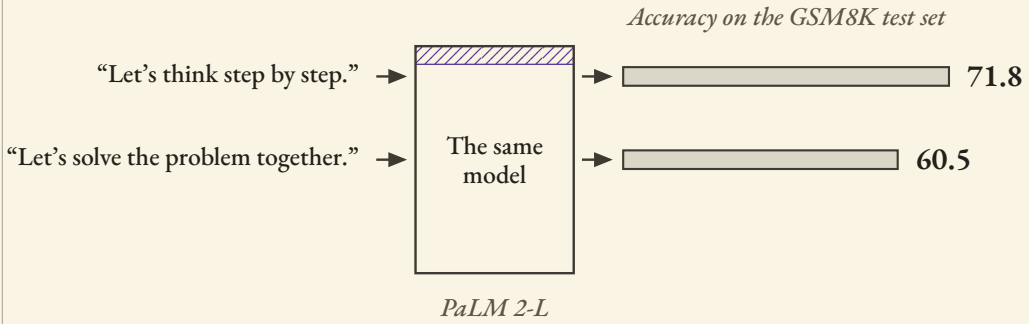
Yang et al. 2024, section 5.2.3, p. 13

Same model, different instruction



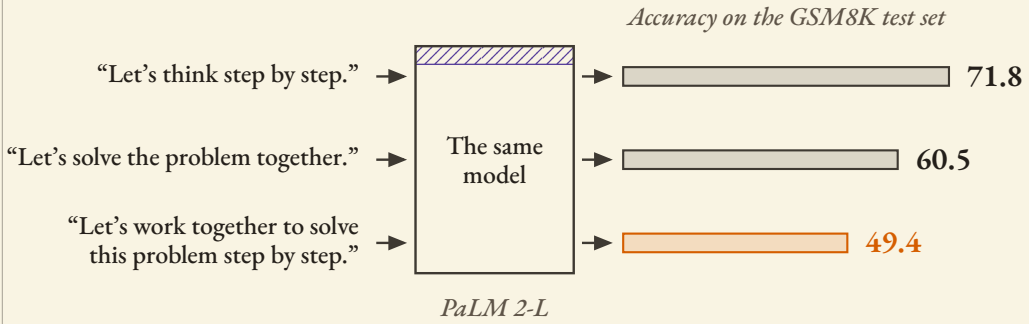
Yang et al. 2024, section 5.2.3, p. 13

Same model, different instruction



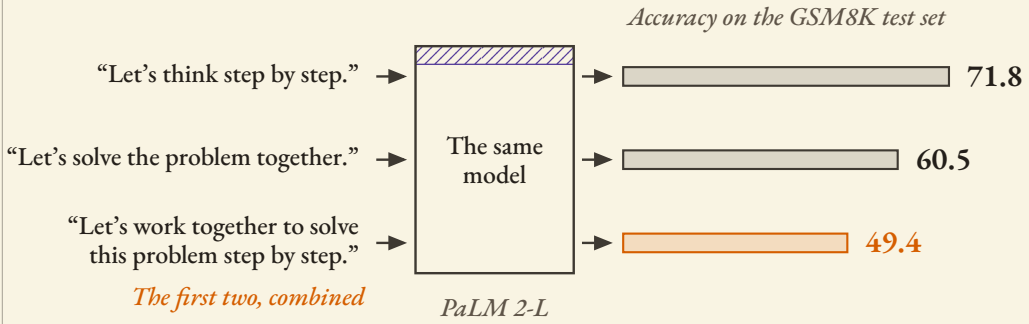
Yang et al. 2024, section 5.2.3, p. 13

Same model, different instruction



Yang et al. 2024, section 5.2.3, p. 13

Same model, different instruction



Yang et al. 2024, section 5.2.3, p. 13

One instruction, before and after GEPA

Agrawal et al. 2026, Fig. 2, Table 2

The seed instruction

`create_query_hop2`

Given the fields `question`, `summary_1`, produce the fields `query`.

One instruction, before and after GEPA

The seed instruction

Agrawal et al. 2026, Fig. 2, Table 2

`create_query_hop2`

Given the fields `question`, `summary_1`, produce the fields `query`.

`create_query_hop2` *GEPA's instruction, GPT-4.1 Mini*

[...] **Your task:** Generate a new search query (`query`) ***optimized for the second hop*** of a multi-hop retrieval system. [...]

Key Observations and Lessons:

- First-hop documents often cover one entity or aspect.
- Remaining relevant documents often involve connected or higher-level concepts mentioned in `summary_1` but not explicitly asked in the original question. [...]
- If `summary_1` describes a population for a small civil parish, but the question wants the total population of the wider region, your query should target that wider region (e.g., "Madeira archipelago population in 2011"). [...]

One instruction, before and after GEPA

Agrawal et al. 2026, Fig. 2, Table 2

The seed instruction

`create_query_hop2`

Given the fields `question`, `summary_1`, produce the fields `query`.

Lessons written by the reflecting model

`create_query_hop2` *GEPA's instruction, GPT-4.1 Mini*

[...] **Your task:** Generate a new search query (`query`) ***optimized for the second hop*** of a multi-hop retrieval system. [...]

Key Observations and Lessons:

- First-hop documents often cover one entity or aspect.
- Remaining relevant documents often involve connected or higher-level concepts mentioned in `summary_1` but not explicitly asked in the original question. [...]
- If `summary_1` describes a population for a small civil parish, but the question wants the total population of the wider region, your query should target that wider region (e.g., "Madeira archipelago population in 2011"). [...]

One instruction, before and after GEPA

Agrawal et al. 2026, Fig. 2, Table 2

The seed instruction

`create_query_hop2`

Given the fields `question`, `summary_1`, produce the fields `query`.

Lessons written by the reflecting model

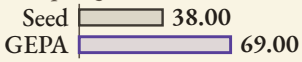
`create_query_hop2` *GEPA's instruction, GPT-4.1 Mini*

[...] **Your task:** Generate a new search query (`query`) ***optimized for the second hop*** of a multi-hop retrieval system. [...]

Key Observations and Lessons:

- First-hop documents often cover one entity or aspect.
- Remaining relevant documents often involve connected or higher-level concepts mentioned in `summary_1` but not explicitly asked in the original question. [...]
- If `summary_1` describes a population for a small civil parish, but the question wants the total population of the wider region, your query should target that wider region (e.g., "Madeira archipelago population in 2011"). [...]

System's test score, HotpotQA, GPT-4.1 Mini



The optimisation problem

An input

$$\Phi(\overset{\downarrow}{x}; \underbrace{\langle \Pi, \Theta \rangle}_{\Pi\text{: all instructions; } \Theta\text{: all weights}})$$

Output y of the system Φ

Agrawal et al. 2026, eq. (1) and (2)

The optimisation problem

Metric, a score in $[0, 1]$

An input

Gold answer, a rubric or unit tests

$$\mu\Big(\Phi(\underbrace{x}_{\downarrow};\underbrace{\langle \Pi, \Theta \rangle}_{\Pi:\ \text{all instructions};\ \Theta:\ \text{all weights}}), \underbrace{m}_{\downarrow}\Big)$$

Output y of the system Φ

Agrawal et al. 2026, eq. (1) and (2)

The optimisation problem

Metric, a score in $[0, 1]$

An input

Gold answer, a rubric or unit tests

$$\mathbb{E}\Big[\, \mu\big(\Phi(x; \underbrace{\langle \Pi, \Theta \rangle}), m\big) \,\Big]$$

Π: all instructions; Θ: all weights

Output y of the system Φ

Over (x, m) drawn from the task distribution

Agrawal et al. 2026, eq. (1) and (2)

The optimisation problem

$\langle \Pi^*, \Theta^* \rangle$ = the instructions and weights that maximise

Metric, a score in $[0, 1]$

An input

Gold answer, a rubric or unit tests

$$\mathbb{E}\Big[\, \mu\big(\Phi(x; \underbrace{\langle \Pi, \Theta \rangle}), m\big) \,\Big] \quad \text{subject to} \quad \#\text{rollouts} \le B$$

Π: all instructions; Θ: all weights

Budget

Output y of the system Φ

Over (x, m) drawn from the task distribution

A rollout: one run of Φ on one input, scored by μ.

Agrawal et al. 2026, eq. (1) and (2)

The optimisation problem

$\langle \Pi^*, \Theta^* \rangle$ = the instructions and weights that maximise

Metric, a score in $[0, 1]$

An input

Gold answer, a rubric or unit tests

$$\mathbb{E}\Big[\, \mu\big(\Phi(x; \langle \Pi, \Theta \rangle), m\big) \,\Big] \quad \text{subject to} \quad \#\text{rollouts} \le B$$

Π: all instructions; Θ: all weights

Budget

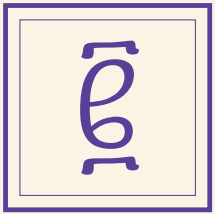
Output y of the system Φ

Over (x, m) drawn from the task distribution

A rollout: one run of Φ on one input, scored by μ.

GEPA: weights frozen, instructions searched

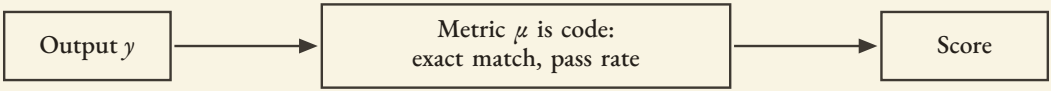
Agrawal et al. 2026, eq. (1) and (2)



Section 2

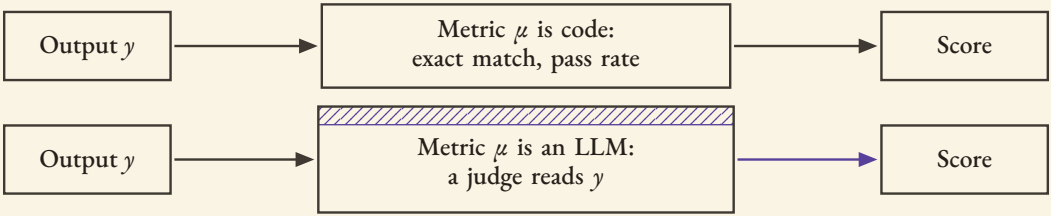
Learning from text

The metric can be an LLM



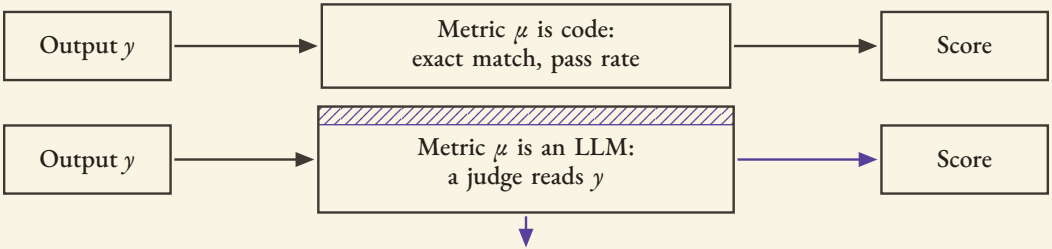
Agrawal et al. 2026, pp. 4, 5, 25; DSPy tutorial "GEPA for Privacy-Conscious Delegation"

The metric can be an LLM



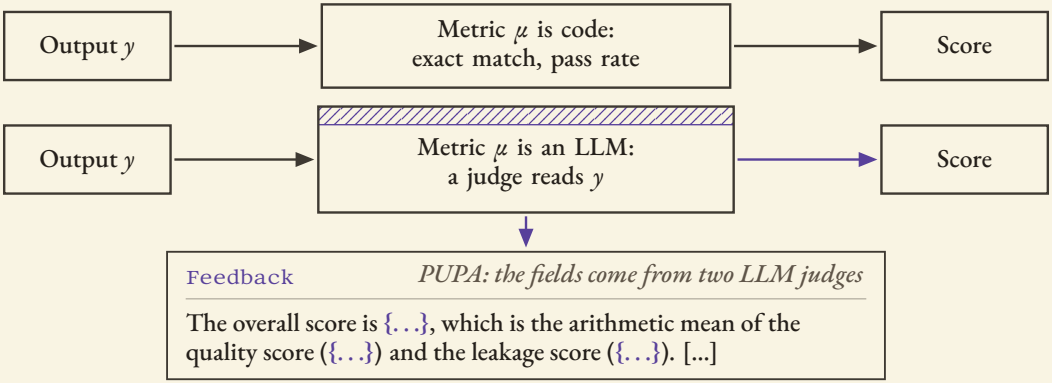
Agrawal et al. 2026, pp. 4, 5, 25; DSPy tutorial "GEPA for Privacy-Conscious Delegation"

The metric can be an LLM



Agrawal et al. 2026, pp. 4, 5, 25; DSPy tutorial "GEPA for Privacy-Conscious Delegation"

The metric can be an LLM



Agrawal et al. 2026, pp. 4, 5, 25; DSPy tutorial "GEPA for Privacy-Conscious Delegation"

A rollout returns a score and text

`system_prompt` *The seed instruction*

You are a helpful assistant. You are given a question and you need to answer it. The answer should be given at the end of your response in exactly the format '### <final answer>'

GEPA's recorded AIME test; Agrawal et al. 2026, pp. 6, 7

A rollout returns a score and text

`system_prompt` *The seed instruction*

You are a helpful assistant. You are given a question and you need to answer it. The answer should be given at the end of your response in exactly the format '### <final answer>'

Problem x (2022 AIME I, problem 6)
Find the number of ordered pairs of integers (a, b) such that the sequence 3, 4, 5, a, b, 30, 40, 50 is strictly increasing and no set of four (not necessarily consecutive) terms forms an arithmetic progression.

GEPA's recorded AIME test; Agrawal et al. 2026, pp. 6, 7

A rollout returns a score and text

`system_prompt` *The seed instruction*

You are a helpful assistant. You are given a question and you need to answer it. The answer should be given at the end of your response in exactly the format '### <final answer>'

Execution trace of GPT-4.1 nano, last line

```
### <final answer> 234
```

Problem x (2022 AIME I, problem 6)
Find the number of ordered pairs of integers (a, b) such that the sequence 3, 4, 5, a, b, 30, 40, 50 is strictly increasing and no set of four (not necessarily consecutive) terms forms an arithmetic progression.
Gold answer m: `### 228`
GEPA's recorded AIME test; Agrawal et al. 2026, pp. 6, 7

A rollout returns a score and text

`system_prompt` *The seed instruction*

You are a helpful assistant. You are given a question and you need to answer it. The answer should be given at the end of your response in exactly the format '### <final answer>'

Problem x (2022 AIME I, problem 6)

Find the number of ordered pairs of integers (a, b) such that the sequence 3, 4, 5, a, b, 30, 40, 50 is strictly increasing and no set of four (not necessarily consecutive) terms forms an arithmetic progression.

Gold answer m: `### 228`

GEPA's recorded AIME test; Agrawal et al. 2026, pp. 6, 7

→

Execution trace of GPT-4.1 nano, last line

`### <final answer> 234`

Metric μ: is `### 228` in the response? No: score 0

A rollout returns a score and text

`system_prompt` *The seed instruction*

You are a helpful assistant. You are given a question and you need to answer it. The answer should be given at the end of your response in exactly the format '### <final answer>'

Problem x (2022 AIME I, problem 6)
Find the number of ordered pairs of integers (a, b) such that the sequence 3, 4, 5, a, b, 30, 40, 50 is strictly increasing and no set of four (not necessarily consecutive) terms forms an arithmetic progression.
Gold answer m: `### 228`

Execution trace of GPT-4.1 nano, last line

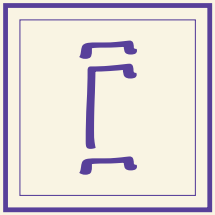
`### <final answer> 234`

Metric μ: is `### 228` in the response? No: score 0
Feedback function μ_f returns the score and this text:
Evaluation trace: what the metric wrote

`Feedback`

The generated response is incorrect. The correct answer is '### 228'. Ensure that the correct answer is included in the response exactly as it is. Here is some additional context that might be helpful: solution: [...]

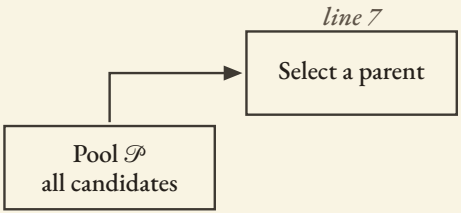
GEPA's recorded AIME test; Agrawal et al. 2026, pp. 6, 7



Section 3

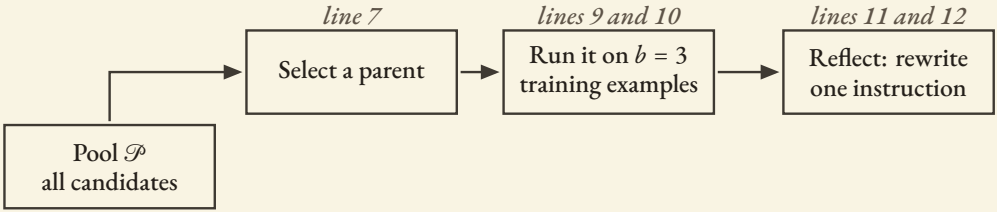
The GEPA loop

GEPA's loop, with the lines of Algorithm 1



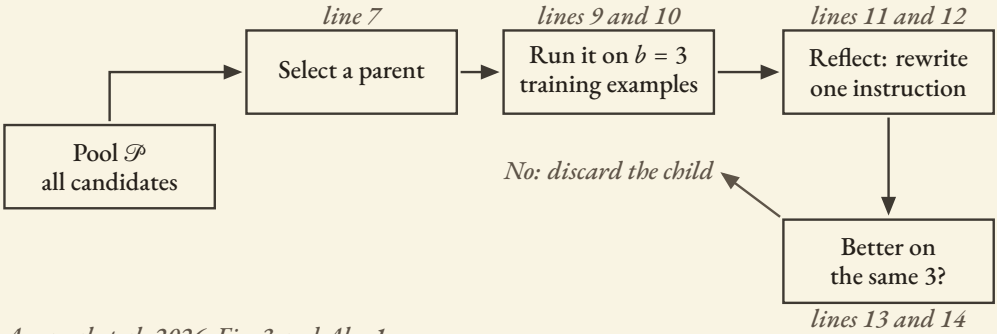
Agrawal et al. 2026, Fig. 3 and Alg. 1

GEPA's loop, with the lines of Algorithm 1



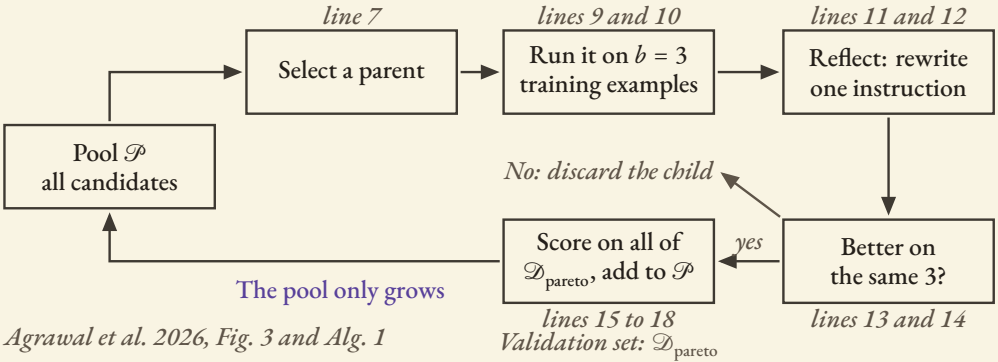
Agrawal et al. 2026, Fig. 3 and Alg. 1

GEPA's loop, with the lines of Algorithm 1



Agrawal et al. 2026, Fig. 3 and Alg. 1

GEPA's loop, with the lines of Algorithm 1



Agrawal et al. 2026, Fig. 3 and Alg. 1

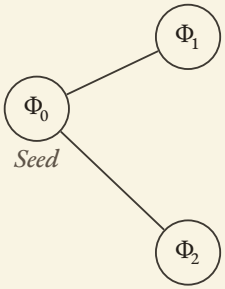
GEPA grows a tree of candidates



Seed

Agrawal et al. 2026, Alg. 1 and Fig. 6

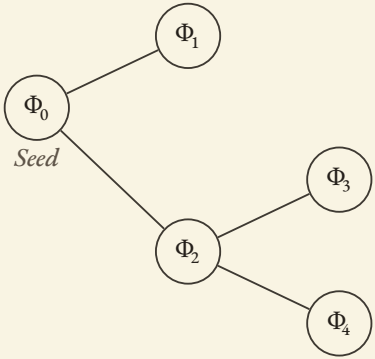
GEPA grows a tree of candidates



Each child is its parent with one instruction rewritten after reflection.

Agrawal et al. 2026, Alg. 1 and Fig. 6

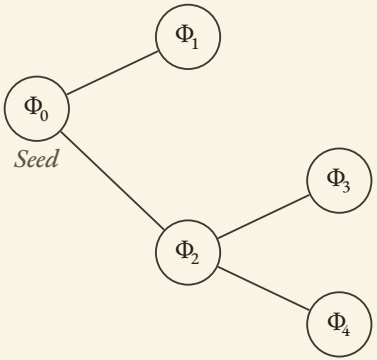
GEPA grows a tree of candidates



Each child is its parent with one instruction rewritten after reflection.

Agrawal et al. 2026, Alg. 1 and Fig. 6

GEPA grows a tree of candidates

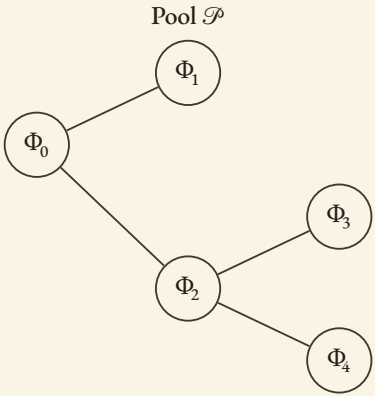


Each child is its parent with one instruction rewritten after reflection.

Pool $\mathcal{P}$:
all the candidates

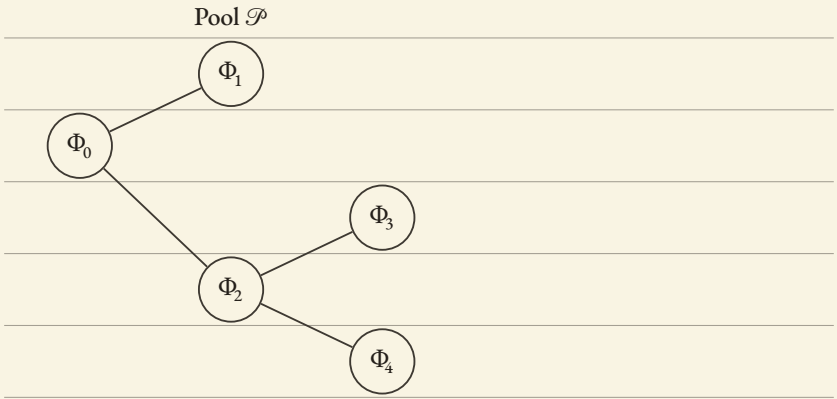
Agrawal et al. 2026, Alg. 1 and Fig. 6

The Pareto front of the pool



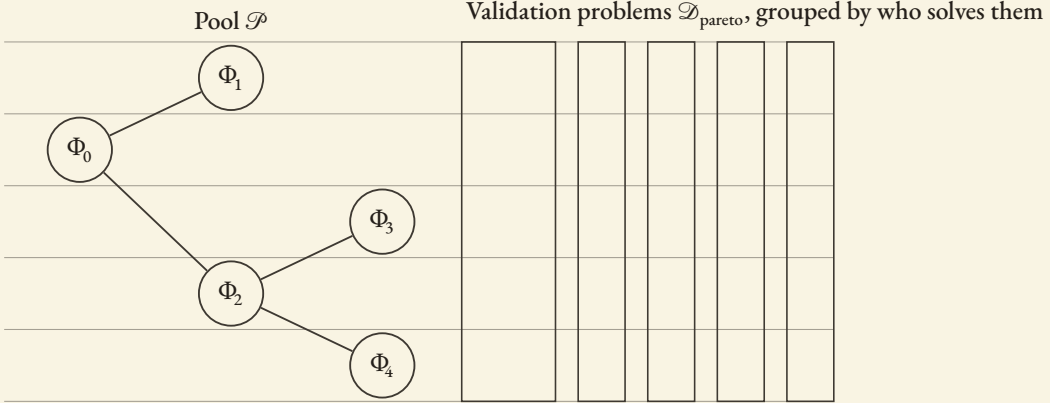
Agrawal et al. 2026, p. 7 and Alg. 2

The Pareto front of the pool



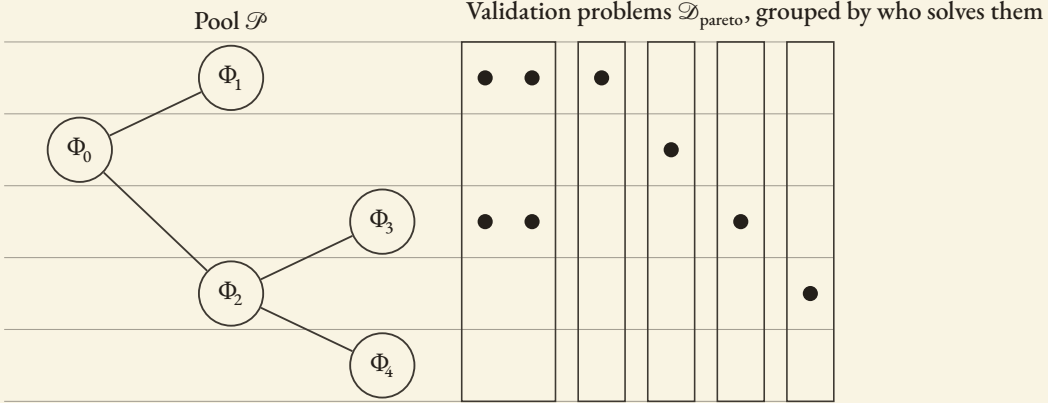
Agrawal et al. 2026, p. 7 and Alg. 2

The Pareto front of the pool



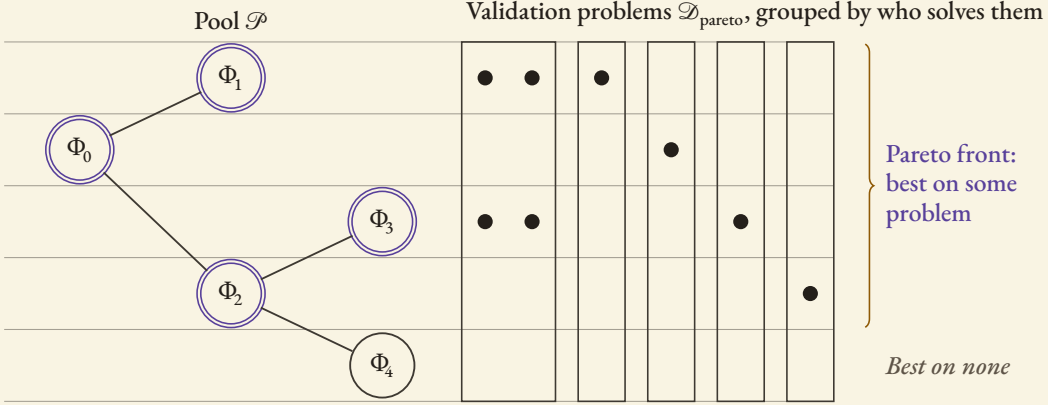
Agrawal et al. 2026, p. 7 and Alg. 2

The Pareto front of the pool



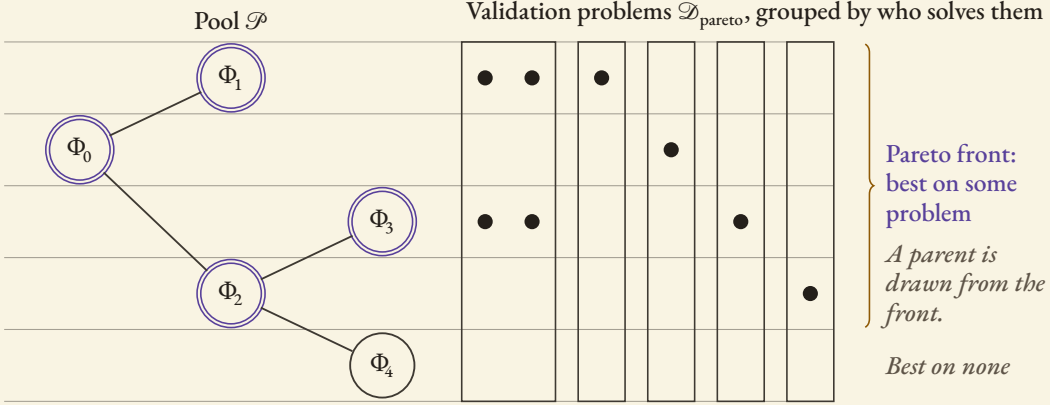
Agrawal et al. 2026, p. 7 and Alg. 2

The Pareto front of the pool



Agrawal et al. 2026, p. 7 and Alg. 2

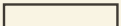
The Pareto front of the pool



Agrawal et al. 2026, p. 7 and Alg. 2

Setup

HotpotQA, questions

$\mathcal{D}_{\text{feedback}}$ *Training split*  150

Minibatches of $b = 3$
Reflection reads inputs, outputs, feedback

Agrawal et al. 2026, Alg. 1, pp. 8, 23, 27

Setup

HotpotQA, questions

$\mathcal{D}_{\text{feedback}}$ *Training split* — 150

$\mathcal{D}_{\text{pareto}}$ *Validation split* — 300

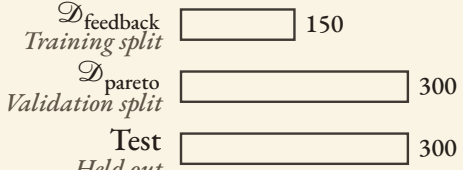
Minibatches of $b = 3$
Reflection reads inputs, outputs, feedback

Each accepted child runs on all
Scores only, to select candidates

Agrawal et al. 2026, Alg. 1, pp. 8, 23, 27

Setup

HotpotQA, questions



Minibatches of $b = 3$
Reflection reads inputs, outputs, feedback

Each accepted child runs on all
Scores only, to select candidates

Reported once, at the end

Agrawal et al. 2026, Alg. 1, pp. 8, 23, 27

Setup

HotpotQA, questions

$\mathcal{D}_{\text{train}}$

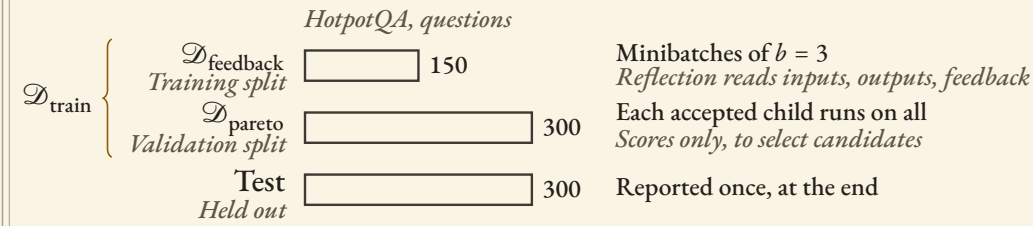
- $\mathcal{D}_{\text{feedback}}$ — *Training split* — 150 — Minibatches of $b = 3$ — *Reflection reads inputs, outputs, feedback*
- $\mathcal{D}_{\text{pareto}}$ — *Validation split* — 300 — Each accepted child runs on all — *Scores only, to select candidates*

Test — *Held out* — 300 — Reported once, at the end

1 **Split $\mathcal{D}_{\text{train}}$ into $\mathcal{D}_{\text{feedback}}$, $\mathcal{D}_{\text{pareto}}$, s.t. $|\mathcal{D}_{\text{pareto}}| = n_{\text{pareto}}$**
2 Initialize candidates $\mathcal{P} \leftarrow [\Phi]$
3 **for each** (x_i, m_i) **in** $\mathcal{D}_{\text{pareto}}$ **do**
4 $\quad S_\Phi[i] \leftarrow \mu(\Phi(x_i), m_i)$

Agrawal et al. 2026, Alg. 1, pp. 8, 23, 27

Setup



1 Split $\mathcal{D}_{\text{train}}$ into $\mathcal{D}_{\text{feedback}}$, $\mathcal{D}_{\text{pareto}}$, s.t. $|\mathcal{D}_{\text{pareto}}| = n_{\text{pareto}}$

▸ 2 **Initialize candidates $\mathcal{P} \leftarrow [\Phi]$** *// Φ: the seed*

▸ 3 **for each (x_i, m_i) in $\mathcal{D}_{\text{pareto}}$ do** *// all 300 questions*

▸ 4 $S_\Phi[i] \leftarrow \mu(\Phi(x_i), m_i)$ *// S_Φ: one row of scores*

Agrawal et al. 2026, Alg. 1, pp. 8, 23, 27

Algorithm 1 in words

Algorithm 1, GEPA

- **while the budget B is not exhausted do**
  - **$\Phi \leftarrow$ select a candidate from the pool $\mathcal{P}$**

  $\mathcal{M} \leftarrow$ sample a few questions from $\mathcal{D}_{\text{feedback}}$
  Use Φ to solve $\mathcal{M}$. Collect feedback F, score σ, traces
  $\Phi' \leftarrow$ update Φ with F, σ and the traces
  Get the score σ' of Φ' on $\mathcal{M}$
  if $\sigma' > \sigma$ **then**
  Add Φ' to $\mathcal{P}$ as a child of Φ

Algorithm 1 in words

Algorithm 1, GEPA

while the budget B is not exhausted **do**
  $\Phi \leftarrow$ select a candidate from the pool $\mathcal{P}$
- $\mathcal{M} \leftarrow$ sample a few questions from $\mathcal{D}_{\text{feedback}}$
- Use Φ to solve $\mathcal{M}$. Collect feedback F, score σ, traces
  $\Phi' \leftarrow$ update Φ with F, σ and the traces
  Get the score σ' of Φ' on $\mathcal{M}$
  if $\sigma' > \sigma$ **then**
    Add Φ' to $\mathcal{P}$ as a child of Φ

Algorithm 1 in words

Algorithm 1, GEPA

while the budget B is not exhausted **do**
  $\Phi \leftarrow$ select a candidate from the pool $\mathcal{P}$
  $\mathcal{M} \leftarrow$ sample a few questions from $\mathcal{D}_{\text{feedback}}$
  Use Φ to solve $\mathcal{M}$. Collect feedback F, score σ, traces
▸ **$\Phi' \leftarrow$ update Φ with F, σ and the traces**
  Get the score σ' of Φ' on $\mathcal{M}$
  if $\sigma' > \sigma$ **then**
    Add Φ' to $\mathcal{P}$ as a child of Φ

Algorithm 1 in words

Algorithm 1, GEPA

while the budget B is not exhausted **do**
  $\Phi \leftarrow$ select a candidate from the pool $\mathcal{P}$
  $\mathcal{M} \leftarrow$ sample a few questions from $\mathcal{D}_{\text{feedback}}$
  Use Φ to solve $\mathcal{M}$. Collect feedback F, score σ, traces
  $\Phi' \leftarrow$ update Φ with F, σ and the traces
- **Get the score σ' of Φ' on $\mathcal{M}$**
- **if $\sigma' > \sigma$ then**
    Add Φ' to $\mathcal{P}$ as a child of Φ

Algorithm 1 in words

Algorithm 1, GEPA

while the budget B is not exhausted **do**
  $\Phi \leftarrow$ select a candidate from the pool $\mathcal{P}$
  $\mathcal{M} \leftarrow$ sample a few questions from $\mathcal{D}_{\text{feedback}}$
  Use Φ to solve $\mathcal{M}$. Collect feedback F, score σ, traces
  $\Phi' \leftarrow$ update Φ with F, σ and the traces
  Get the score σ' of Φ' on $\mathcal{M}$
  if $\sigma' > \sigma$ **then**
  - **Add Φ' to $\mathcal{P}$ as a child of Φ**

Algorithm 1: one iteration

Algorithm 1, GEPA *Agrawal et al. 2026, p. 6; GEPA library*

```
6  while budget B not exhausted do                          // B in rollouts
7    k ← SelectCandidate(𝒫, S)                              // draw a parent
8    j ← SelectModule(Φ_k)                                  // which instruction, round robin
9    ℳ ← minibatch of size b from 𝒟_feedback
10   Gather feedback, scores, traces for Φ_k[j] on ℳ using μ_f
11   π'_j ← UpdatePrompt(π_j, feedbacks, traces[j])
12   Φ' ← Copy of Φ_k w/ module j updated by π'_j
13   σ, σ' ← avg score on ℳ (before, after)
14   if σ' improved then
15       Add Φ' to 𝒫; Add k to 𝒜
16       for each (x_i, m_i) in 𝒟_pareto: S_Φ'[i] ← μ(Φ'(x_i), m_i)
21 return Φ* maximizing average score on 𝒟_pareto
```

Algorithm 1: one iteration

Algorithm 1, GEPA *Agrawal et al. 2026, p. 6; GEPA library*

```
   while budget B not exhausted do                                  // B in rollouts
     k ← SelectCandidate(𝒫, S)                                      // draw a parent
     j ← SelectModule(Φ_k)                                          // which instruction, round robin
▸    𝓜 ← minibatch of size b from 𝒟_feedback                        // b = 3 training examples
▸    Gather feedback, scores, traces for Φ_k[j] on 𝓜 using μ_f      // parent rerun, with traces
     π'_j ← UpdatePrompt(π_j, feedbacks, traces[j])
     Φ' ← Copy of Φ_k w/ module j updated by π'_j
     σ, σ' ← avg score on 𝓜 (before, after)
     if σ' improved then
         Add Φ' to 𝒫; Add k to 𝒜
         for each (x_i, m_i) in 𝒟_pareto: S_Φ'[i] ← μ(Φ'(x_i), m_i)
   return Φ* maximizing average score on 𝒟_pareto
```

Algorithm 1: one iteration

Algorithm 1, GEPA *Agrawal et al. 2026, p. 6; GEPA library*

```
6  while budget B not exhausted do                                  // B in rollouts
7    k ← SelectCandidate(𝒫, S)                                      // draw a parent
8    j ← SelectModule(Φ_k)                                          // which instruction, round robin
9    ℳ ← minibatch of size b from 𝒟_feedback                        // b = 3 training examples
10   Gather feedback, scores, traces for Φ_k[j] on ℳ using μ_f      // parent rerun, with traces
11 ▸ π'_j ← UpdatePrompt(π_j, feedbacks, traces[j])                 // an LLM rewrites instruction π_j
12 ▸ Φ' ← Copy of Φ_k w/ module j updated by π'_j                   // the child
13   σ, σ' ← avg score on ℳ (before, after)
14   if σ' improved then
15     Add Φ' to 𝒫; Add k to 𝒜
16     for each (x_i, m_i) in 𝒟_pareto: S_Φ'[i] ← μ(Φ'(x_i), m_i)
21 return Φ* maximizing average score on 𝒟_pareto
```

Algorithm 1: one iteration

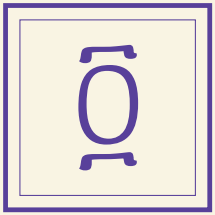
Algorithm 1, GEPA *Agrawal et al. 2026, p. 6; GEPA library*

```
while budget B not exhausted do                                   // B in rollouts
    k ← SelectCandidate(𝒫, S)                                     // draw a parent
    j ← SelectModule(Φ_k)                                         // which instruction, round robin
    ℳ ← minibatch of size b from 𝒟_feedback                       // b = 3 training examples
    Gather feedback, scores, traces for Φ_k[j] on ℳ using μ_f     // parent rerun, with traces
    π'_j ← UpdatePrompt(π_j, feedbacks, traces[j])                // an LLM rewrites instruction π_j
    Φ' ← Copy of Φ_k w/ module j updated by π'_j                  // the child
    σ, σ' ← avg score on ℳ (before, after)                        // same b examples
    if σ' improved then                                           // strictly better
        Add Φ' to 𝒫; Add k to 𝒜                                   // 𝒫: pool; 𝒜: parents
        for each (x_i, m_i) in 𝒟_pareto: S_Φ'[i] ← μ(Φ'(x_i), m_i)
return Φ* maximizing average score on 𝒟_pareto
```

Algorithm 1: one iteration

Algorithm 1, GEPA *Agrawal et al. 2026, p. 6; GEPA library*

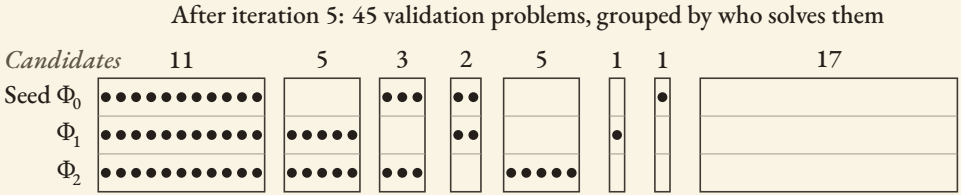
```
6  while budget B not exhausted do                                  // B in rollouts
7    k ← SelectCandidate(𝒫, S)                                      // draw a parent
8    j ← SelectModule(Φ_k)                                          // which instruction, round robin
9    ℳ ← minibatch of size b from 𝒟_feedback                        // b = 3 training examples
10   Gather feedback, scores, traces for Φ_k[j] on ℳ using μ_f      // parent rerun, with traces
11   π'_j ← UpdatePrompt(π_j, feedbacks, traces[j])                 // an LLM rewrites instruction π_j
12   Φ' ← Copy of Φ_k w/ module j updated by π'_j                   // the child
13   σ, σ' ← avg score on ℳ (before, after)                         // same b examples
14   if σ' improved then                                            // strictly better
15     Add Φ' to 𝒫; Add k to 𝒜                                      // 𝒫: pool; 𝒜: parents
▸ 16   for each (x_i, m_i) in 𝒟_pareto: S_Φ'[i] ← μ(Φ'(x_i), m_i)   // S_Φ': all validation scores
▸ 21 return Φ* maximizing average score on 𝒟_pareto                 // best average
```



Section 4

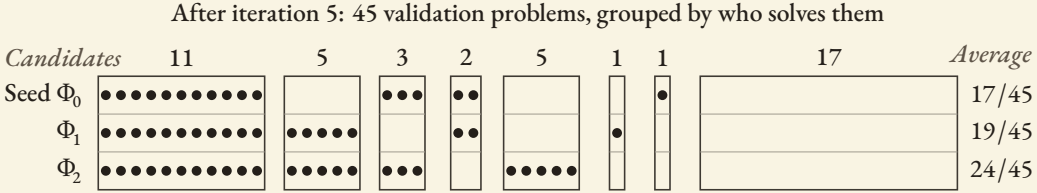
Choosing the parent

The score matrix of a real run



DSPy tutorial "GEPA for AIME": GPT-4.1 Mini solves, GPT-5 reflects

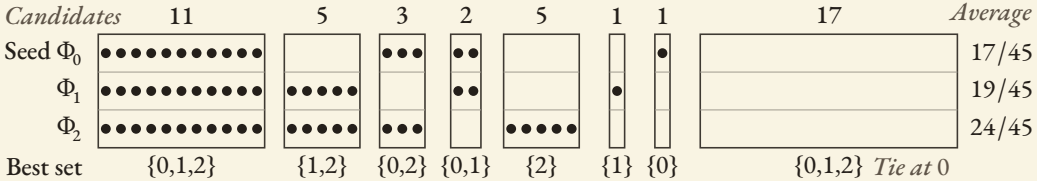
The score matrix of a real run



DSPy tutorial "GEPA for AIME": GPT-4.1 Mini solves, GPT-5 reflects

The score matrix of a real run

After iteration 5: 45 validation problems, grouped by who solves them

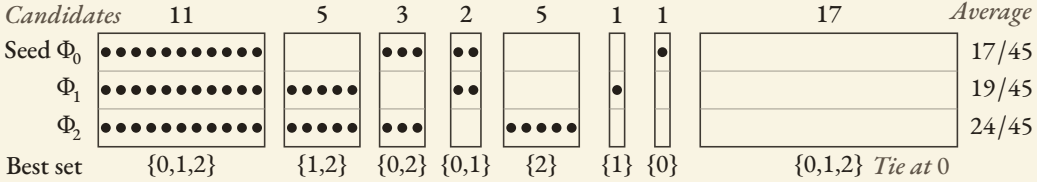


Φ is best on problem *i* when
no candidate scores higher on *i*

DSPy tutorial "GEPA for AIME": GPT-4.1 Mini solves, GPT-5 reflects

The score matrix of a real run

After iteration 5: 45 validation problems, grouped by who solves them



Φ is best on problem *i* when no candidate scores higher on *i*

Dominance: as high on every problem, and higher on one

DSPy tutorial "GEPA for AIME": GPT-4.1 Mini solves, GPT-5 reflects

Algorithm 2 in words

Algorithm 2, Pareto-based candidate selection

- **function SelectCandidate($\mathcal{P}$)**
  - **$\mathcal{C} \leftarrow \emptyset$**
  - **for each problem q in the validation set $\mathcal{D}_{\text{pareto}}$ do**
    - **Add to $\mathcal{C}$ all Φ in the pool $\mathcal{P}$ that best answer q**

  Remove from $\mathcal{C}$ any Φ dominated by another. What is left: $\hat{\mathcal{C}}$

  For Φ in $\hat{\mathcal{C}}$, let $f(\Phi)$ be the number of problems where Φ gives the best answer

  Sample Φ from $\hat{\mathcal{C}}$ with probability proportional to $f(\Phi)$

Algorithm 2 in words

Algorithm 2, Pareto-based candidate selection

function SELECTCANDIDATE($\mathcal{P}$)
  $\mathcal{C} \leftarrow \emptyset$
  for each problem q in the validation set $\mathcal{D}_{\text{pareto}}$ **do**
    Add to $\mathcal{C}$ all Φ in the pool $\mathcal{P}$ that best answer q
- **Remove from $\mathcal{C}$ any Φ dominated by another. What is left: $\hat{\mathcal{C}}$**

  For Φ in $\hat{\mathcal{C}}$, let $f(\Phi)$ be the number of problems where Φ gives the best answer
  Sample Φ from $\hat{\mathcal{C}}$ with probability proportional to $f(\Phi)$

Algorithm 2 in words

Algorithm 2, Pareto-based candidate selection

function SELECTCANDIDATE($\mathcal{P}$)
  $\mathcal{C} \leftarrow \emptyset$
  for each problem q in the validation set $\mathcal{D}_{\text{pareto}}$ **do**
    Add to $\mathcal{C}$ all Φ in the pool $\mathcal{P}$ that best answer q
  Remove from $\mathcal{C}$ any Φ dominated by another. What is left: $\hat{\mathcal{C}}$

- For Φ in $\hat{\mathcal{C}}$, let $f(\Phi)$ be the number of problems where Φ gives the best answer
- Sample Φ from $\hat{\mathcal{C}}$ with probability proportional to $f(\Phi)$

Algorithm 2 on the matrix

Algorithm 2, Pareto-based candidate selection *Agrawal et al. 2026; DSPy tutorial*

```
function SelectCandidate(𝒫, S)                          // the run's pool, scores
    for each i do                                       // i: 45 validation problems
        s*[i] ← max_k S_{𝒫[k]}[i]                       // top score on i
        𝒫*[i] ← {𝒫[k] : S_{𝒫[k]}[i] = s*[i]}            // best sets, shown before
    𝒞 ← unique candidates in ⋃_i 𝒫*[i]
    D ← ∅
    while there exists Φ ∈ 𝒞 \ D dominated by another in 𝒞 \ D do
        D ← D ∪ {Φ}
    Remove D from each 𝒫*[i] to get 𝒫̂*[i]
    Let f[Φ] = number of i for which Φ ∈ 𝒫̂*[i]
    Sample Φ_k from 𝒞̂ with probability ∝ f[Φ_k]
    return index k of Φ_k in 𝒫
```

Algorithm 2 on the matrix

Algorithm 2, Pareto-based candidate selection *Agrawal et al. 2026; DSPy tutorial*

```
1  function SelectCandidate(𝒫, S)                          // the run's pool, scores
3      for each i do                                       // i: 45 validation problems
4          s*[i] ← max_k S_𝒫[k][i]                         // top score on i
5          𝒫*[i] ← {𝒫[k] : S_𝒫[k][i] = s*[i]}              // best sets, shown before
▸ 7    𝒞 ← unique candidates in ⋃_i 𝒫*[i]                  // Pareto front: Φ0, Φ1, Φ2
8      D ← ∅
9      while there exists Φ ∈ 𝒞 \ D dominated by another in 𝒞 \ D do
10         D ← D ∪ {Φ}
12     Remove D from each 𝒫*[i] to get 𝒫̂*[i]
13     Let f[Φ] = number of i for which Φ ∈ 𝒫̂*[i]
14     Sample Φ_k from 𝒞̂ with probability ∝ f[Φ_k]
15     return index k of Φ_k in 𝒫
```

Algorithm 2 on the matrix

Algorithm 2, Pareto-based candidate selection *Agrawal et al. 2026; DSPy tutorial*

function SELECTCANDIDATE($\mathcal{P}, S$) *// the run's pool, scores*
  for each i **do** *// i: 45 validation problems*
    $s^*[i] \leftarrow \max_k S_{\mathcal{P}[k]}[i]$ *// top score on i*
    $\mathcal{P}^*[i] \leftarrow \{\mathcal{P}[k] : S_{\mathcal{P}[k]}[i] = s^*[i]\}$ *// best sets, shown before*
  $\mathscr{C} \leftarrow$ unique candidates in $\bigcup_i \mathcal{P}^*[i]$ *// Pareto front: Φ_0, Φ_1, Φ_2*
▸ $D \leftarrow \emptyset$ *// D: the dominated*
▸ **while there exists** $\Phi \in \mathscr{C} \setminus D$ **dominated by another in** $\mathscr{C} \setminus D$ **do** *// none here*
▸ $D \leftarrow D \cup \{\Phi\}$ *// $\hat{\mathscr{C}} = \mathscr{C} \setminus D$*
  Remove D from each $\mathcal{P}^*[i]$ to get $\hat{\mathcal{P}}^*[i]$
  Let $f[\Phi]$ = number of i for which $\Phi \in \hat{\mathcal{P}}^*[i]$
  Sample Φ_k from $\hat{\mathscr{C}}$ with probability $\propto f[\Phi_k]$
  return index k of Φ_k in $\mathcal{P}$

Algorithm 2 on the matrix

Algorithm 2, Pareto-based candidate selection *Agrawal et al. 2026; DSPy tutorial*

```
function SelectCandidate(𝒫, S)                          // the run's pool, scores
    for each i do                                       // i: 45 validation problems
        s*[i] ← max_k S_{𝒫[k]}[i]                       // top score on i
        𝒫*[i] ← {𝒫[k] : S_{𝒫[k]}[i] = s*[i]}            // best sets, shown before
    𝒞 ← unique candidates in ⋃_i 𝒫*[i]                  // Pareto front: Φ0, Φ1, Φ2
    D ← ∅                                               // D: the dominated
    while there exists Φ ∈ 𝒞 \ D dominated by another in 𝒞 \ D do   // none here
        D ← D ∪ {Φ}                                     // Ĉ = 𝒞 \ D
    Remove D from each 𝒫*[i] to get P̂*[i]               // nothing removed
    Let f[Φ] = number of i for which Φ ∈ P̂*[i]          // f = 34, 36, 41
    Sample Φ_k from Ĉ with probability ∝ f[Φ_k]
    return index k of Φ_k in 𝒫
```

Algorithm 2 on the matrix

Algorithm 2, Pareto-based candidate selection *Agrawal et al. 2026; DSPy tutorial*

1 **function** SELECTCANDIDATE($\mathcal{P}, S$) *// the run's pool, scores*
3 **for each** i **do** *// i: 45 validation problems*
4 $s^*[i] \leftarrow \max_k S_{\mathcal{P}[k]}[i]$ *// top score on i*
5 $\mathcal{P}^*[i] \leftarrow \{\mathcal{P}[k] : S_{\mathcal{P}[k]}[i] = s^*[i]\}$ *// best sets, shown before*
7 $\mathscr{C} \leftarrow$ unique candidates in $\bigcup_i \mathcal{P}^*[i]$ *// Pareto front: Φ_0, Φ_1, Φ_2*
8 $D \leftarrow \emptyset$ *// D: the dominated*
9 **while** there exists $\Phi \in \mathscr{C} \setminus D$ dominated by another in $\mathscr{C} \setminus D$ **do** *// none here*
10 $D \leftarrow D \cup \{\Phi\}$ *// $\hat{\mathscr{C}} = \mathscr{C} \setminus D$*
12 Remove D from each $\mathcal{P}^*[i]$ to get $\hat{\mathcal{P}}^*[i]$ *// nothing removed*
13 Let $f[\Phi]$ = number of i for which $\Phi \in \hat{\mathcal{P}}^*[i]$ *// f = 34, 36, 41*
▸ 14 **Sample Φ_k from $\hat{\mathscr{C}}$ with probability $\propto f[\Phi_k]$** *// 34/111, 36/111, 41/111*
▸ 15 **return index k of Φ_k in $\mathcal{P}$** *// run drew Φ_1, then Φ_2*

GEPA against GRPO and MIPROv2, per task

Qwen3 8B, test score (%)

	HotpotQA	IFBench	HoVer	PUPA	AIME 2025	LiveBench Math	Aggregate
Baseline	42.33	36.90	35.33	80.82	27.33	48.70	45.23

Agrawal et al. 2026, Table 1; Opsahl-Ong et al. 2024

GEPA against GRPO and MIPROv2, per task

Qwen3 8B, test score (%)

	HotpotQA	IFBench	HoVer	PUPA	AIME 2025	LiveBench Math	Aggregate
Baseline	42.33	36.90	35.33	80.82	27.33	48.70	45.23
GRPO	43.33	35.88	38.67	86.66	38.00	51.26	48.91
Rollouts, GRPO	24,000	24,000	24,000	24,000	24,000	24,000	

Agrawal et al. 2026, Table 1; Opsahl-Ong et al. 2024

GEPA against GRPO and MIPROv2, per task

Qwen3 8B, test score (%)

	HotpotQA	IFBench	HoVer	PUPA	AIME 2025	LiveBench Math	Aggregate
Baseline	42.33	36.90	35.33	80.82	27.33	48.70	45.23
GRPO	43.33	35.88	38.67	86.66	38.00	51.26	48.91
MIPROv2	55.33	36.22	47.33	81.55	20.00	46.60	47.84
Instructions and examples, chosen by Bayesian search							
Rollouts, GRPO	24,000	24,000	24,000	24,000	24,000	24,000	

Agrawal et al. 2026, Table 1; Opsahl-Ong et al. 2024

GEPA against GRPO and MIPROv2, per task

Qwen3 8B, test score (%)

	HotpotQA	IFBench	HoVer	PUPA	AIME 2025	LiveBench Math	Aggregate
Baseline	42.33	36.90	35.33	80.82	27.33	48.70	45.23
GRPO	43.33	35.88	38.67	86.66	38.00	51.26	48.91
MIPROv2	55.33	36.22	47.33	81.55	20.00	46.60	47.84
Instructions and examples, chosen by Bayesian search							
GEPA	62.33	38.61	52.33	91.85	32.00	51.95	54.85
Rollouts, GRPO	24,000	24,000	24,000	24,000	24,000	24,000	
Rollouts, GEPA	6,871	3,593	7,051	2,426	1,839	1,839	

Agrawal et al. 2026, Table 1; Opsahl-Ong et al. 2024

GEPA against GRPO and MIPROv2, per task

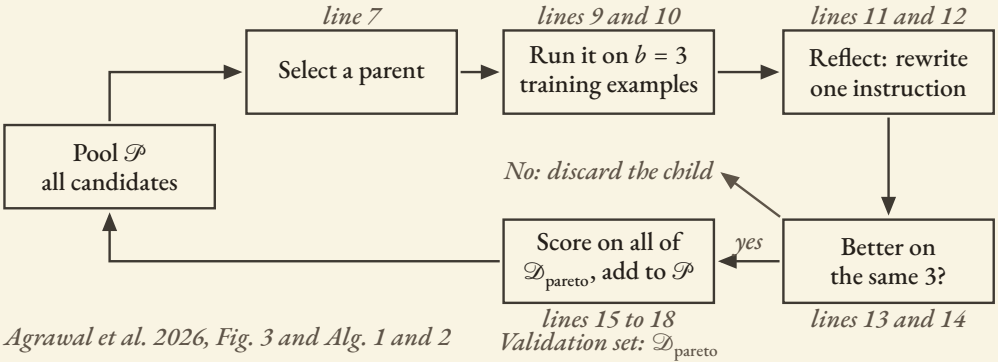
Qwen3 8B, test score (%)

GRPO ahead: 38.00 against 32.00

	HotpotQA	IFBench	HoVer	PUPA	AIME 2025	LiveBench Math	Aggregate
Baseline	42.33	36.90	35.33	80.82	27.33	48.70	45.23
GRPO	43.33	35.88	38.67	86.66	**38.00**	51.26	48.91
MIPROv2	55.33	36.22	47.33	81.55	20.00	46.60	47.84
Instructions and examples, chosen by Bayesian search							
GEPA	62.33	38.61	52.33	91.85	32.00	51.95	54.85
Rollouts, GRPO	24,000	24,000	24,000	24,000	24,000	24,000	
Rollouts, GEPA	6,871	3,593	7,051	2,426	1,839	1,839	

Agrawal et al. 2026, Table 1; Opsahl-Ong et al. 2024

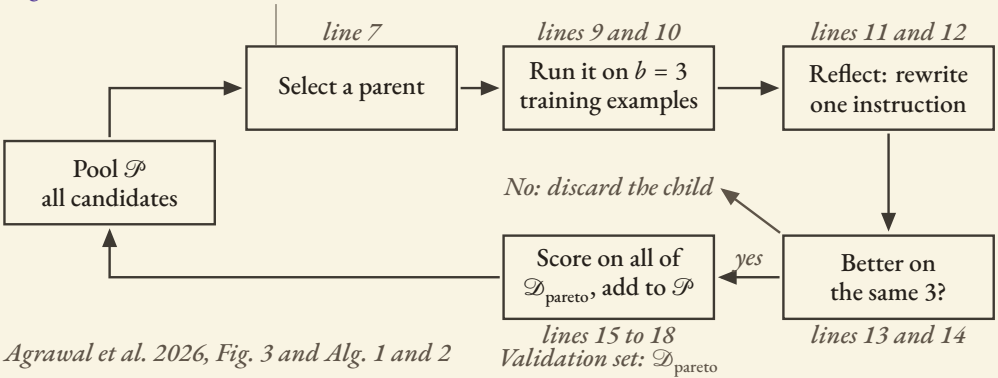
GEPA, assembled



Agrawal et al. 2026, Fig. 3 and Alg. 1 and 2

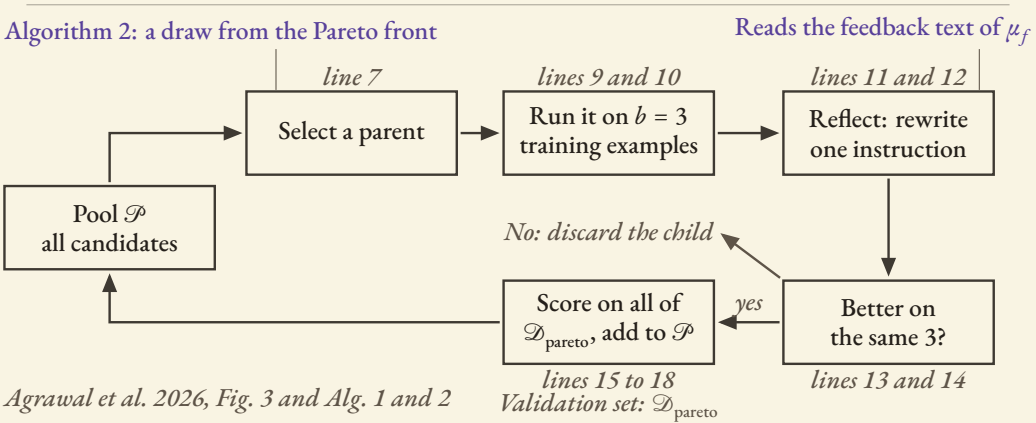
GEPA, assembled

Algorithm 2: a draw from the Pareto front



Agrawal et al. 2026, Fig. 3 and Alg. 1 and 2

GEPA, assembled



Agrawal et al. 2026, Fig. 3 and Alg. 1 and 2

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia and Omar Khattab. GEPA: reflective prompt evolution can outperform reinforcement learning. *ICLR*, 2026 (oral). arxiv.org/abs/2507.19457; code github.com/gepa-ai/gepa and github.com/gepa-ai/gepa-artifact

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou and Xinyun Chen. Large language models as optimizers. *ICLR*, 2024. arxiv.org/abs/2309.03409

Karl Cobbe and others. Training verifiers to solve math word problems (GSM8K), 2021. arxiv.org/abs/2110.14168

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. *EMNLP*, 2024. arxiv.org/abs/2406.11695

Zhihong Shao and others. DeepSeekMath: pushing the limits of mathematical reasoning in open language models, 2024. arxiv.org/abs/2402.03300

References, continued

Zhilin Yang and others. HotpotQA: a dataset for diverse, explainable multi-hop question answering. *EMNLP*, 2018. aclanthology.org/D18-1259

Siyan Li, Vethavikashini Chithrra Raghuram, Omar Khattab, Julia Hirschberg and Zhou Yu. PAPILLON: privacy preservation from Internet-based and local language model ensembles (the PUPA task). *NAACL*, 2025. aclanthology.org/2025.naacl-long.173

Runs quoted in this lecture: the GEPA library's recorded AIME test, `tests/test_aime_prompt_optimization` at commit `d771eb2`; the DSPy tutorials "GEPA for AIME", a run of 12 August 2025, and "GEPA for Privacy-Conscious Delegation", dspy.ai/tutorials/gepa_aime and dspy.ai/tutorials/gepa_papillon